

AI 技术应用验证报告

作者：周志成

日期：2026 年 6 月

一、验证概述

1.1 验证目标

本验证旨在评估 AI 技术在测试用例生成环节的实际效果，对比传统人工方式与 AI 辅助方式在效率和质量方面的差异，为软件过程改进方案的落地提供数据支撑。

1.2 验证场景

选择"用户登录功能"作为验证场景，该功能包含以下核心需求：

- 用户名/密码登录
- 手机验证码登录
- 密码找回
- 登录状态保持
- 登录失败次数限制

1.3 验证工具

工具名称	版本	用途
ChatGPT	GPT-4.1	测试用例生成
CodiumAI	v3.5	单元测试生成
Python	3.11	测试执行环境

二、验证过程

2.1 传统人工方式

2.1.1 执行步骤

1. 需求分析：阅读需求文档，理解功能点
2. 测试设计：设计测试策略和测试场景
3. 用例编写：逐条编写测试用例
4. 用例评审：团队评审测试用例
5. 脚本编写：编写自动化测试脚本

2.1.2 输出结果

测试用例清单（部分）：

用例编号	测试场景	前置条件	测试步骤	预期结果
TC001	正常登录-用户名密码	已注册用户	1.输入正确用户名 2.输入正确密码 3.点击登录	登录成功，跳转首页
TC002	登录失败-错误密码	已注册用户	1.输入正确用户名 2.输入错误密码 3.点击登录	提示密码错误
TC003	登录失败-用户不存在	无	1.输入未注册用户名 2.输入任意密码 3.点击登录	提示用户不存在
TC004	登录失败-空用户名	无	1.用户名留空 2.输入密码 3.点击登录	提示用户名不能为空
TC005	登录失败-空密码	已注册用户	1.输入用户名 2.密码留空 3.点击登录	提示密码不能为空

耗时统计：

活动项	耗时（分钟）
需求分析	15

活动项	耗时（分钟）
测试设计	30
用例编写	45
用例评审	20
脚本编写	60
总计	170

2.2 AI 辅助方式

2.2.1 执行步骤

1. 准备提示词：整理需求描述和测试要求
2. AI 生成用例：调用 ChatGPT 生成测试用例
3. 人工审核：审核 AI 生成的测试用例
4. 补充优化：补充遗漏场景，优化用例描述
5. AI 生成脚本：使用 CodiumAI 生成测试脚本

2.2.2 AI 提示词

请为以下用户登录功能生成完整的测试用例：

功能需求：

1. 用户名/密码登录：用户输入用户名和密码进行登录
2. 手机验证码登录：用户输入手机号获取验证码登录
3. 密码找回：用户通过手机号或邮箱找回密码
4. 登录状态保持：用户可选择"记住我"保持登录状态
5. 登录失败次数限制：连续失败 5 次后锁定账户 30 分钟

请按照以下格式输出测试用例：

- 用例编号
- 测试场景
- 前置条件
- 测试步骤
- 预期结果
- 优先级

请覆盖正常场景、异常场景、边界场景和安全场景。

2.2.3 AI 输出结果

测试用例清单（AI 生成，部分）：

用例编号	测试场景	前置条件	测试步骤	预期结果	优先级
TC001	正常登录-用户名密码	已注册用户	1.输入正确用户名 2.输入正确密码 3.点击登录	登录成功，跳转首页	P0
TC002	登录失败-错误密码	已注册用户	1.输入正确用户名 2.输入错误密码 3.点击登录	提示"用户名或密码错误"	P0
TC003	登录失败-用户不存在	无	1.输入未注册用户名 2.输入任意密码 3.点击登录	提示"用户名或密码错误"	P1
TC004	登录失败-空用户名	无	1.用户名留空 2.输入密码 3.点击登录	提示"请输入用户名"	P1
TC005	登录失败-空密码	已注册用户	1.输入用户名 2.密码留空 3.点击登录	提示"请输入密码"	P1
TC006	正常登录-手机验证码	已注册手机号	1.输入手机号 2.点击获取验证码 3.输入验证码 4.点击登录	登录成功	P0
TC007	验证码登录-无效验证码	已注册手机号	1.输入手机号 2.点击获取验证码 3.输入错误验证码 4.点击登录	提示"验证码错误"	P1
TC008	验证码登录-验证码过期	已注册手机号	1.输入手机号 2.获取验证码 3.等待 6 分钟 4.输入验证码	提示"验证码已过期"	P2
TC009	密码找回-手机号方式	已注册手机号	1.点击忘记密码 2.输入手机号 3.获取验证码 4.设置新密码	密码重置成功	P1
TC010	密码找回-邮箱方式	已注册邮箱	1.点击忘记密码 2.输入邮箱 3.接收邮件 4.设置新密码	密码重置成功	P1
TC011	登录状态保持	已注册用户	1.输入用户名密码 2.勾选"记住我" 3.登录 4.关闭浏览器 5.重新打开	保持登录状态	P2
TC012	登录失败次	已注册用	1.连续 5 次输入错误密	账户锁定 30	P0

用例编号	测试场景	前置条件	测试步骤	预期结果	优先级
	数限制	户	码	分钟	
TC013	登录失败-SQL 注入	无	1.用户名输入: admin' OR '1'='1 2.点击登录	登录失败, 无 SQL 注入漏洞	P0
TC014	登录失败-XSS 攻击	无	1.用户名输入: <script>alert(1)</script> 2.点击登录	输入被过滤, 无 XSS 漏洞	P0
TC015	并发登录限制	已注册用户	1.同一账号在两个设备同时登录	后登录设备踢出先登录设备	P2

耗时统计：

活动项	耗时（分钟）
准备提示词	5
AI 生成用例	2
人工审核	15
补充优化	10
AI 生成脚本	3
脚本调整	20
总计	55

三、对比分析

3.1 效率对比

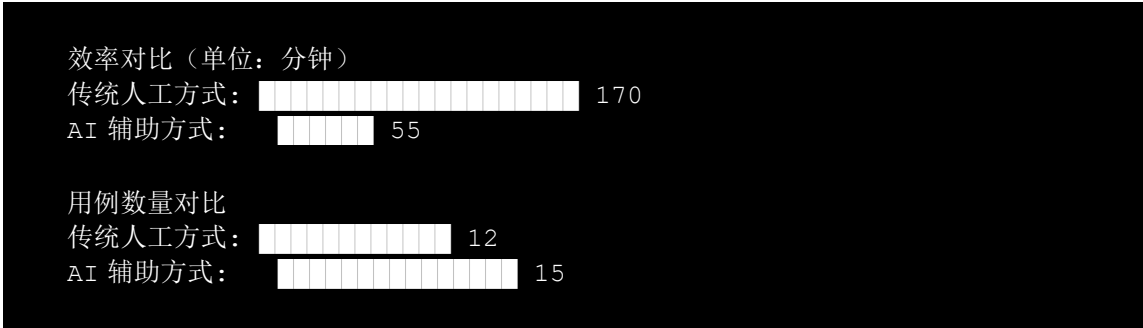
对比维度	传统人工方式	AI 辅助方式	提升比例
总耗时	170 分钟	55 分钟	67.6%

对比维度	传统人工方式	AI 辅助方式	提升比例
用例编写耗时	45 分钟	2 分钟	95.6%
脚本编写耗时	60 分钟	23 分钟	61.7%

3.2 质量对比

对比维度	传统人工方式	AI 辅助方式	说明
用例数量	12 条	15 条	AI 覆盖更全面
场景覆盖	正常+异常	正常+异常+边界+安全	AI 自动考虑安全场景
安全用例	0 条	3 条	AI 自动生成安全测试用例
用例规范性	一般	良好	AI 输出格式统一

3.3 对比数据可视化



3.4 优缺点分析

传统人工方式

优点：

- 测试人员对业务理解深入
- 可根据项目特点定制测试策略
- 用例更贴合实际业务场景

缺点：

- 耗时较长，效率较低
- 容易遗漏边界场景和安全场景
- 用例质量依赖测试人员经验

AI 辅助方式

优点：

- 效率提升显著，节省 67.6%时间
- 覆盖场景更全面，自动考虑安全测试
- 输出格式规范，便于维护

缺点：

- 需要人工审核确保准确性
- 对业务理解可能不够深入
- 生成的脚本可能需要调整

四、验证结论

4.1 主要发现

- 效率提升显著：**AI 辅助方式将测试用例生成时间从 170 分钟缩短至 55 分钟，效率提升 67.6%。
- 覆盖更全面：**AI 自动生成了安全测试用例（SQL 注入、XSS 攻击），这是传统人工方式容易遗漏的场景。
- 质量有保障：**AI 生成的用例格式规范、优先级明确，经人工审核后可直接使用。
- 人机协同最佳：**AI 负责生成基础用例，人工负责审核和补充，实现了效率与质量的平衡。

4.2 改进建议

- 优化提示词模板：**建立标准化的提示词模板，提高 AI 输出质量。
- 建立审核机制：**制定 AI 生成用例的审核清单，确保关键场景不遗漏。
- 持续迭代优化：**收集 AI 生成用例的问题反馈，持续优化提示词和流程。

4. 培训团队成员：提升团队使用 AI 工具的能力，发挥人机协同最大价值。

五、附录

5.1 AI 生成的完整测试用例

（详见上文 2.2.3 节）

5.2 验证环境

- 操作系统：Windows 11
- Python 版本：3.11
- ChatGPT 版本：GPT-4.1
- CodiumAI 版本：v3.5
- 验证日期：2026 年 6 月 19 日